



GenomeIQ — Testing Guide

A complete walk-through for testing **every function, page, and endpoint** in GenomeIQ. Each test has:

- **Goal** — what you're verifying
- **Steps** — exact clicks / commands
- **Expected** — what success looks like

Before you start: make sure the server is running. Press **F5** in VS Code with the *Run GenomeIQ Web App* config. Browser should open at <http://127.0.0.1:8000>.



Test Sequences You'll Use

Save these in a notepad — you'll copy-paste them many times.

Test sequence A — BRCA1 fragment (expected: **Cancer**)

```
ATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAGTGTCCCATCTGTC
TGGAGTTGATCAAGGAACCTGTCTCCACAAAGTGTGACCACATATTTTGCAAATTTTGCATGCTGAAACTTCTCAACCAGAA
GAAAGGGCCTTCACAGTGTCTTTATGTAAGAATGATATAACCAAAAGGG
```

Test sequence B — INS fragment (expected: **Diabetes**)

```
ATGGCCCTGTGGATGCGCCTCCTGCCCTGCTGGCGCTGCTGGCCCTCTGGGGACCTGACCCAGCCGCAGCCTTTGTGAACC
AACACCTGTGCGGCTCACACCTGGTGGAAAGCTCTCTACCTAGTGTGCGGGGAACGAGGCTTCTTCTACACACCCAAGACCCG
CCGGGAGGCAGAGGACCTGCAGGTGGGGCAGGTGGAGCTGGGCGGTGGCCCTGGTGCAGGCAGCCTGCAGCCCTTGGCCCTG
GAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGT
```

Test sequence C — APP fragment (expected: **Alzheimers**)

```
ATGCTGCCCGTTTTGGCACTGCTCCTGCTGGCCGCCTGGACGGCTCGGGCAGTGCAGAATTCTGACATGCTGCAGAATTTCA
GCCAGAATCAACCCGACTTCTCTGACTATGACAACAGCCACAGCAGCCAGCCAGCCAGCCATGGAAGATGAGGATGAAGA
CGAAGATGAGGAAGATGAAGACGAGCCAAAGAGGAAGATGAACCCAGGAAGGAGGAT
```

Test sequence D — ACTB fragment (expected: **Normal**)

```
ATGGATGATGATATCGCCGCGCTCGTCGTCGACAACGGCTCCGGCATGTGCAAAGCCGGCTTCGCGGGCGACGATGCCCCGA
GGGCACTCTTCCAGCCTTCCTTCTGGGCATGGAGTCTGTGGCATCCACGAACTACCTTCAACTCCATCATGAAGTGTGA
CGTGGACATCCGCAAAGACCTGTACGCCAACACACATGAAGATCAAGATCATTGCTCCTCCTGAGCGCAAGTACTCCGTGTG
GATCGGCGGCTCCATCCTGGCCTCGCTGTCCACCTTCCAG
```

Test sequence E — Invalid (should fail validation)

```
HELLOWORLD123
```

Test sequence F — Too short (should fail length check)

```
ATG
```

Test sequence G — Random low-complexity (should trigger HIGH OOD risk)

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```



Part 1 — UI Feature Tests



Page 1: Analyze

1.1 Live sequence statistics

 **Goal:** verify the live counters update as you type.

 Steps:

1. Open the **Analyze** tab.
2. Click the input textarea.
3. Type or paste sequence A character-by-character (or chunk).

 Expected:

- "Length" counter updates to **213**.
- "GC%" updates to **~39%**.
- "Valid" turns **Yes** (green) once length ≥ 20 .

1.2 Clear button

 **Goal:** verify the textarea can be emptied.

 Steps:

1. Paste any sequence.
2. Click **Clear**.

 **Expected:** textarea empty, Length = 0, GC% = "—", Valid = No.

1.3 Sample buttons (all 4)

 **Goal:** every sample button loads its corresponding test sequence.

 **Steps:** click each sample button one by one:

- **BRCA1 (Cancer)**
- **INS (Diabetes)**
- **APP (Alzheimer's)**
- **ACTB (Normal)**

 **Expected:** textarea fills with the matching sample (compare with sequences A–D above), live stats update.

1.4 Model backbone selector

 **Goal:** the segmented control switches between TF-IDF / DNABERT-2 / Ensemble.

 **Steps:**

1. Look at the "Model backbone" pill group.
2. Click each option.

 **Expected:** active pill highlighted in white/brand color. If DNABERT-2 head wasn't trained, it appears grayed-out — that's correct.

1.5 Analysis options checkboxes

 **Goal:** all three optional toggles work.

 **Steps:**

1. Toggle **Saliency explanation** off → run analysis → result should NOT have a saliency map card.
2. Toggle it back on → run again → saliency card returns.
3. Same for **Out-of-distribution** and **Similar sequence search**.

 **Expected:** corresponding cards appear/disappear in the results area.

1.6 Validation error — invalid characters

 **Goal:** invalid characters get rejected with a friendly message.

 **Steps:**

1. Paste sequence E (**HELLLOWORLD123**).
2. Click **Analyze Sequence**.

 **Expected:** red error toast: "No valid DNA bases found." or similar.

1.7 Validation error — too short

 **Goal:** short sequences are rejected.

 **Steps:** paste sequence F (ATG) → Analyze.

 **Expected:** error: "Sequence too short. Minimum 20 bases required."

1.8 Full prediction — Cancer (TF-IDF)

 **Goal:** sequence A predicts Cancer using the default backbone.

 **Steps:**

1. Click **BRCA1 (Cancer)** sample.
2. Make sure backbone = **TFIDF**.
3. Click **Analyze Sequence**.

 **Expected:**

- Verdict: **Cancer**, ~79% confidence
 - Probability bars show Cancer leading
 - Confidence gauge (radial) shows ~79%
 - Saliency map renders with colored bases
 - Composition doughnut shows A/T/G/C/N counts
 - Circular plot renders
 - "Closest Known Sequences" lists 5 hits (mostly Cancer)
 - ORFs may or may not be present (depends on sequence)
-

1.9 Full prediction — Diabetes

 **Goal:** sequence B predicts Diabetes.

 **Steps:** load INS sample → Analyze.

 **Expected:** verdict = **Diabetes**, confidence ~96%.

1.10 Full prediction — Alzheimers

 **Goal:** sequence C predicts Alzheimer's.

 **Steps:** load APP sample → Analyze.

 **Expected:** verdict = **Alzheimers**, confidence ~44%. OOD banner likely says **HIGH RISK** because APP region differs from training.

1.11 Full prediction — Normal

 **Goal:** sequence D predicts Normal.

 **Steps:** load ACTB sample → Analyze.

✓ **Expected:** verdict = **Normal**, confidence ~79%, perfect OOD risk = LOW (or MEDIUM).

1.12 Backbone comparison

🎯 **Goal:** the same input gives different results across backbones.

📖 **Steps:**

1. Load BRCA1 sample.
2. Run with **TFIDF** → note confidence (~79%).
3. Run with **DNABERT-2** → note confidence (~85%).
4. Run with **Ensemble** → note confidence (~82%).

✓ **Expected:** all three return Cancer but with slightly different confidence values. The "predicted_class.model" badge in the verdict card shows the active backbone.

1.13 OOD high-risk detection

🎯 **Goal:** out-of-distribution detector flags repetitive sequences.

📖 **Steps:** paste sequence G (AAA. . .) → Analyze.

✓ **Expected:** OOD banner shows **HIGH RISK / unreliable**. Some prediction will still appear but with the warning.

1.14 Saliency map rendering

🎯 **Goal:** colored DNA bases display correctly.

📖 **Steps:** run prediction with sequence A → scroll to saliency card.

✓ **Expected:**

- DNA characters in monospace font
 - Each character has a background colored from slate (low) → amber (medium) → rose (high)
 - Below the map: a color-scale legend
 - "Top Regions" chips show positions with highest mean score
-

1.15 Composition chart

🎯 **Goal:** doughnut chart renders A/T/G/C/N counts.

📖 **Steps:** after any prediction, look at the "Sequence Composition" card.

✓ **Expected:** Chart.js doughnut with 5 segments (A blue, T rose, G green, C amber, N gray), legend at bottom, theme-aware (light/dark colors).

1.16 Circular plot

 **Goal:** Plotly polar plot renders.

 **Steps:** after any prediction, scroll to "Circular Sequence Map".

 **Expected:**

- Outer ring: blue line tracing GC content along the sequence
 - Middle ring: colored markers (saliency hotspots)
 - Inner rings: thick green / purple arcs for ORFs (forward / reverse strand)
 - Center: predicted class name + confidence
 - Hover over arcs/markers shows tooltip
-

1.17 Closest Known Sequences

 **Goal:** BLAST-lite retrieval works.

 **Steps:** after BRCA1 prediction, look at the "Closest Known Sequences" card.

 **Expected:** 5 ranked hits, each with:

- Disease badge (color-coded)
 - Rank number
 - Similarity percentage
 - Truncated sequence preview (monospace)
 - Source ID
-

1.18 ORF table

 **Goal:** ORFs detected on both strands and three frames.

 **Steps:** load INS or ACTB samples (both have ORFs) → Analyze → look at "Open Reading Frames" card.

 **Expected:** table rows showing:

- Strand (+ or -)
- Frame (0, 1, or 2)
- Start–end positions
- Protein length in amino acids

If no ORFs found (e.g. very short sequence), "No ORFs of significant length found." message appears.

1.19 PDF download

 **Goal:** PDF report generates and downloads.

 **Steps:**

1. Run any prediction.
2. Look at top-right header → click the blue **PDF** button.

✓ Expected:

- File downloads as `genomeiq-report-<timestamp>.pdf`
 - Open it: shows verdict, OOD banner, probability table, composition stats, similar hits, ORFs, top saliency regions, sequence fragment, disclaimer
 - Author "MD Naimur Rashid" in footer
 - File size ~4-8 KB
-

💬 Page 2: Copilot

2.1 Empty-state suggestions

🎯 **Goal:** suggestion buttons load on an empty chat.

📖 **Steps:** click 💬 **Copilot** in sidebar (don't type anything).

✓ Expected:

- 5 clickable suggestion buttons appear:
 - "What is BRCA1 and how does it relate to cancer?"
 - "Explain the role of APP and PSEN1 in Alzheimer's disease."
 - "What does the OOD detector mean by high-risk?"
 - "Why might my Diabetes prediction have low confidence?"
 - "How does GenomelQ's classifier work?"
 - Provider dropdown shows DEMO / GEMINI / OLLAMA with availability indicators
-

2.2 Demo provider chat

🎯 **Goal:** offline template responses work.

📖 **Steps:**

1. Set provider to **DEMO**.
2. Click suggestion "What is BRCA1...".

✓ Expected:

- User bubble appears (right side, brand color)
 - Loading dots animate briefly
 - Assistant bubble (left side, gray) shows formatted response
 - Below the response: source chips (BRCA1, BRCA2, Cancer overview, etc.) with similarity scores
-

2.3 Gemini provider chat

🎯 **Goal:** Gemini 2.5 Flash returns natural-language answers.

📖 **Steps:**

1. Make sure `.env` has a valid `GEMINI_API_KEY`.

2. Set provider to **GEMINI**.
3. Ask: "Explain TP53 in 3 sentences."

✅ **Expected:**

- Response within ~2-5 seconds
 - 3-sentence concise answer about TP53
 - Source chips: TP53, Cancer overview
 - No "GEMINI_API_KEY not set" error
-

2.4 Source chips

🎯 **Goal:** retrieved facts are visible to the user.

📖 **Steps:** any chat answer.

✅ **Expected:** chips below assistant bubble, each labeled <title> · <similarity%>. Hover shows full content.

2.5 Prediction context awareness

🎯 **Goal:** Copilot uses your last analyze result as context.

📖 **Steps:**

1. Go to Analyze, run a Cancer prediction (BRCA1 sample).
2. Switch to Copilot.
3. Ask: "Why is my prediction Cancer?"

✅ **Expected:**

- Right rail shows "Active prediction context: Cancer"
 - Answer references your specific 79% Cancer prediction
 - Does not say "I don't know your prediction"
-

2.6 Provider auto-fallback

🎯 **Goal:** if a provider becomes unavailable, dropdown disables it.

📖 **Steps:**

1. Stop Ollama (or never start it).
2. Reload the page.

✅ **Expected:** OLLAMA option in dropdown shows "(unavailable)" and is disabled.

2.7 Clear chat

🎯 **Goal:** Clear button empties the message list.

Steps:

1. Send a few messages.
2. Click **Clear** in the chat header.

✓ **Expected:** all messages disappear, empty-state and suggestion buttons reappear.

2.8 Free-form question

 **Goal:** typing a custom question works.

 **Steps:** type "What is the difference between BRCA1 and BRCA2?" → Enter.

✓ **Expected:** comparison answer with both gene chips as sources.

2.9 Out-of-knowledge question

 **Goal:** Copilot honestly admits when it doesn't know.

 **Steps:** ask "What is the price of bitcoin today?"

✓ **Expected:** demo mode says no match found; Gemini mode says it can't help with that, refuses to invent.

2.10 Medical-advice refusal

 **Goal:** Copilot refuses to give individualized medical advice.

 **Steps:** ask "Should I take medication X for my BRCA1 mutation?"

✓ **Expected:** Gemini mode politely refuses, explains GenomeIQ is research-only, suggests consulting a clinician.

Page 3: Mutation Lab

3.1 Demo pair

 **Goal:** the "Load demo pair" button populates both boxes.

 **Steps:**

1. Click  **Mutation Lab**.
2. Click **Load demo pair**.

✓ **Expected:** both reference and variant boxes filled with sequences differing at one position.

3.2 Compare sequences

 **Goal:** comparison runs and shows mismatches.

 **Steps:** after loading the demo pair, click **Compare Sequences**.

 **Expected:**

- 4 stat cards: Aligned length / Mismatches / Mutation rate / Length match
 - Two prediction cards (reference vs variant)
 - Probability shift bars (bidirectional, green for + / red for -)
 - Mutation events table with columns: Pos / Ref / Alt / Codon / AA / Effect
 - Effect badges color-coded (synonymous=green, missense=amber, nonsense=red)
-

3.3 Validation error — empty boxes

 **Goal:** missing inputs are caught.

 **Steps:** clear both boxes → click **Compare Sequences**.

 **Expected:** error: "Both reference and variant sequences are required."

3.4 Length mismatch

 **Goal:** different-length sequences still produce a result with a warning.

 **Steps:**

- Reference: paste sequence A (213 chars)
- Variant: paste only the first 100 chars of sequence A
- Compare.

 **Expected:**

- "Length match" card shows **No** (amber)
 - Aligned length = 100
 - Probability shift section computes anyway
-

3.5 Custom mutation

 **Goal:** user-edited variant produces correct effect classification.

 **Steps:**

- Reference: sequence A
- Variant: sequence A with the 4th character changed from **G** to **T** (creating ATG → ATT codon at position 1)
- Compare.

 **Expected:**

- 1 mismatch at position 4
- Codon change **ATG** → **ATT**
- AA change **M** → **I**

- Effect: **missense** (amber badge)

Page 4: Batch Upload

4.1 Create a test FASTA

 **Goal:** prepare a multi-sequence input.

 **Steps:** create a file `test.fasta` containing:

```
>seq_brca1
ATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGCTATTAATGCTATGCAGAAAATCTTAGAGTGTCCCATCTGTC
TGGAGTTGATCAAGGAACCTGTCTCCACAAAGTGTGACCACATATTTTGCAATTTTGCATGCTGAAACTTCTCAACCAGAA
GAAAGGGCCTTCACAGTGTCTTTATGTAAGAATGATATAACCAAAAGGG
>seq_ins
ATGGCCCTGTGGATGCGCCTCCTGCCCCTGCTGGCGCTGCTGGCCCTCTGGGGACCTGACCCAGCCGCAGCCTTTGTGAACC
AACACCTGTGCGGCTCACACCTGGTGAAGCTCTCTACCTAGTGTGCGGGGAACGAGGCTTCTTCTACACACCCAAGACCCG
CCGGGAGGCAGAGGACCTGCAGGTGGGGCAGGTGGAGCTGGGCGGTGGCCCTGGTGCAGGCAGCCTGCAGCCCTTGGCCCTG
GAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGT
>seq_app
ATGCTGCCCGTTTGGCACTGCTCCTGCTGGCCGCCTGGACGGCTCGGGCAGTGCAGAATTCTGACATGCTGCAGAATTTCA
GCCAGAATCAACCCGACTTCTCTGACTATGACAACAGCCACAGCAGCCAGCCAGAGCCAGCCATGGAAGATGAGGATGAAGA
CGAAGATGAGGAAGATGAAGACGAGCCCAAAGAGGAAGATGAACCCAGGAAGGAGGAT
>seq_actb
ATGGATGATGATATCGCCGCGCTCGTCGTCGACAACGGCTCCGGCATGTGCAAAGCCGGCTTCGCGGGCGACGATGCCCCGA
GGGCACTCTTCCAGCCTTCCTTCTGGGCATGGAGTCTGTGGCATCCACGAACTACCTTCAACTCCATCATGAAGTGTGA
CGTGACATCCGCAAAGACCTGTACGCCAACACACATGAAGATCAAGATCATTGCTCCTCCTGAGCGCAAGTACTCCGTGTG
GATCGGCGGCTCCATCCTGGCCTCGCTGTCCACCTTCCAG
```

4.2 Upload + classify

 **Goal:** all 4 records processed.

 **Steps:**

1. Click  **Batch Upload**.
2. Click the file picker → select `test.fasta`.

 **Expected:**

- Three stat cards: Total = 4, Successful = 4, Failed = 0
- Table with 4 rows:
 - `seq_brca1` → Cancer
 - `seq_ins` → Diabetes
 - `seq_app` → Alzheimers
 - `seq_actb` → Normal
- All confidence values visible
- All status = OK (green)

4.3 Mixed-validity FASTA

 **Goal:** invalid records are reported but don't break the run.

 **Steps:** add a fifth record with invalid bases:

```
>seq_bad  
HELLOWORLDXYZ
```

 **Expected:** Total = 5, Successful = 4, Failed = 1. The bad row shows status = "No valid DNA bases found." in red.

Page 5: Knowledge Base

5.1 Disease cards

 **Goal:** all 4 cards render with full info.

 **Steps:** click  **Knowledge Base** in sidebar.

 **Expected:** 4 cards (Cancer / Diabetes / Alzheimers / Normal), each with:

- Top border in disease color
 - Icon, name, short description
 - Long description paragraph
 - "Key genes" chips (BRCA1, BRCA2, etc.)
 - "Pathways" chips (DNA damage response, etc.)
 - "References" with clickable NCBI links
-

5.2 Gene click-through

 **Goal:** NCBI links open in a new tab.

 **Steps:** click any reference link (e.g., the BRCA1 NCBI link).

 **Expected:** new tab opens to ncbi.nlm.nih.gov/gene/672.

Page 6: User Manual

6.1 Quick-start card

 **Goal:** the 6-step quickstart visible at top.

 **Steps:** click  **User Manual** in sidebar.

 **Expected:** purple/blue gradient card titled "Quick start in 60 seconds" with numbered list.

6.2 Collapsible sections

 **Goal:** all 8 feature sections expand and collapse.

 **Steps:** click each `<summary>` (the bold heading row).

 **Expected:** content slides open below; clicking again collapses it.

Page 7: Presentation

7.1 First slide

 **Goal:** title slide loads.

 **Steps:** click  **Presentation** in sidebar.

 **Expected:** "GenomeIQ — DNA Disease Classifier" title, your name in brand color, slide counter shows "1 / 12".

7.2 Next/Previous buttons

 **Goal:** navigation works.

 **Steps:** click **Next** → repeatedly until end, then ← **Previous** back.

 **Expected:** all 12 slides cycle smoothly. Buttons disable at start (Previous) and end (Next). Dot indicators update.

7.3 Keyboard navigation

 **Goal:** arrow keys work.

 **Steps:** with Presentation page focused, press → and ← keys.

 **Expected:** slides advance/retreat. Spacebar also advances. Keys do nothing on other pages.

7.4 Dot navigation

 **Goal:** clicking a dot jumps to that slide.

 **Steps:** click the 6th dot in the row.

 **Expected:** jumps directly to slide 6 (Models table).

Page 8: About

 **Goal:** project info + author credit visible.

 **Steps:** click  **About**.

 Expected:

- Description of GenomeIQ
 - Bullet list of features
 - 4-tile stats grid (CV F1, k-mer config, train samples, classes)
 - "Created by **MD Naimur Rashid**" card with avatar
 - Yellow/amber disclaimer box at the bottom
-

 Theme Toggle

 **Goal:** dark mode works across all pages.

 Steps:

1. Click the sun/moon icon in the top-right.
2. Navigate through every page.

 Expected:

- All pages re-render with dark theme (dark navy background, light text)
 - Charts and circular plot adapt their colors
 - Choice persists after page reload (localStorage)
-

 API Status Indicator

 **Goal:** sidebar bottom shows API connection.

 **Steps:** look at the bottom of the sidebar.

 **Expected:** green dot + "API online" + version number (e.g. 0.2.0). If you stop the server, it goes red + "API offline".

 Part 2 — API Endpoint Tests

Open a PowerShell terminal in the project root and run these commands. The server must be running on port 8000.

All commands assume PowerShell. Replace single-quotes with double-quotes on cmd.exe.

Test 2.1 — Health check

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

 Expected:

```
{
  "status": "ok",
  "version": "0.2.0",
  "classes": ["Alzheimers", "Cancer", "Diabetes", "Normal"]
}
```

Test 2.2 — List models

```
Invoke-RestMethod http://127.0.0.1:8000/models
```

✓ **Expected:** JSON with `tfidf`, `dnabert2`, `ensemble` and their `available` flags.

Test 2.3 — Disease info

```
Invoke-RestMethod http://127.0.0.1:8000/diseases
Invoke-RestMethod http://127.0.0.1:8000/diseases/Cancer
```

✓ **Expected:** array of 4 objects, then a single Cancer object with `key_genes`, `pathways`, `references`.

Test 2.4 — Validate sequence

```
$body = @{ sequence = "ATGGATTTATCTGCTCTT" } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/validate -ContentType
"application/json" -Body $body
```

✓ **Expected:** `is_valid = true` (after 20 char if longer; this 18-char sequence will be `is_valid = false`).

Test 2.5 — Predict

```
$body = @{
  sequence =
  "ATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAGTGTCCTCATCTGT
  CTGGAGTTGATCAAGGAACCTGTCTCCACAAAGTGTGACCACATATTTTGCAAATTTGCATGCTGAAACTTCTCAACCAGA
  AGAAAGGGCCTTCACAGTGTCTTTATGTAAGAATGATATAACCAAAAGGG"
  model = "tfidf"
  top_k_similar = 3
} | ConvertTo-Json
$res = Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/predict -
  ContentType "application/json" -Body $body
```

```
$resp.prediction
$resp.ood
$resp.similar | Select-Object rank, disease, similarity
```

✅ **Expected:** prediction.predicted_class = "Cancer", confidence ~0.79, OOD risk visible, top 3 similar sequences listed.

Test 2.6 — Mutation comparison

```
$body = @{
  reference = "ATGGATTATCTGCTCTTCGCGTTGAAGAAGTACAA"
  variant   = "ATGCATTATCTGCTCTTCGCGTTGAAGAAGTACAA"
} | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/mutation -ContentType
"application/json" -Body $body
```

✅ **Expected:** total_mismatches = 1, position = 4, ref_base = G, alt_base = C, effect = "missense".

Test 2.7 — Batch upload

```
# (after creating test.fasta as in section 4.1)
$form = @{ file = Get-Item .\test.fasta }
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/batch -Form $form
```

✅ **Expected:** total = 4, successful = 4, results array with each prediction.

Test 2.8 — PDF download

```
$body = @{ sequence =
"ATGGATTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAG" } |
ConvertTo-Json
Invoke-WebRequest -Method Post -Uri http://127.0.0.1:8000/report/pdf -ContentType
"application/json" -Body $body -OutFile test_report.pdf
Get-Item test_report.pdf
```

✅ **Expected:** test_report.pdf created, ~5 KB, opens as a valid PDF in any reader.

Test 2.9 — Chat status


```
Invoke-RestMethod http://127.0.0.1:8000/chat/status
```

✓ **Expected:** providers object with demo/gemini/ollama availability + knowledge_base.facts_total.

Test 2.10 — Chat (demo)

```
$body = @{  
    message = "What is BRCA1?"  
    provider = "demo"  
    top_k = 3  
} | ConvertTo-Json  
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/chat -ContentType  
"application/json" -Body $body
```

✓ **Expected:** answer string + retrieved array with BRCA1 fact at top.

Test 2.11 — Chat (gemini)

```
$body = @{  
    message = "Explain APP in one sentence."  
    provider = "gemini"  
} | ConvertTo-Json  
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/chat -ContentType  
"application/json" -Body $body
```

✓ **Expected:** natural-language one-sentence answer about APP gene.

Test 2.12 — Interactive Swagger

📖 **Steps:** open <http://127.0.0.1:8000/docs> in browser.

✓ **Expected:** all endpoints listed; "Try it out" button works on each.



Part 3 — CLI Tests

Test 3.1 — Predict from CLI

```
venv\Scripts\activate  
python -m src.predict --sequence  
"ATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAGTGTCCTCTGT"
```

```
CTGGAGTTGATCAAGGAACCTGTCTCCACAAAGTGTGACCACATATTTTGCAAATTTTGCATGCTGAAACTTCTCAACCAGA  
AGAAAGGGCCTTCACAGTGTCTTTATGTAAGAATGATATAACCAAAAGGG"
```

✓ **Expected:** JSON dump of full prediction payload printed to stdout.

Test 3.2 — Predict from FASTA

```
python -m src.predict --file .\test.fasta
```

✓ **Expected:** batch JSON with 4 prediction entries.

Test 3.3 — Dataset audit

```
python -m src.data.audit
```

✓ **Expected:** JSON report with per_class stats and warnings about imbalance.

Test 3.4 — Build dataset

```
python -m src.data.build_dataset
```

✓ **Expected:** writes `data/processed/train.parquet`, `val.parquet`, `build_summary.json`. Prints summary with ~22,848 `train_balanced_total`.

Test 3.5 — Train TF-IDF

```
python -m src.train
```

✓ **Expected:** trains in ~30 sec, prints classification report with macro F1 ~0.94, saves artifacts to `models/`.

Test 3.6 — Cross-validation

```
python -m src.eval cv --n-splits 5
```

✓ **Expected:** runs 5 folds, prints per-fold metrics, saves `cv_report_k6.json` with `macro_f1_mean` ~0.96.

Test 3.7 — Grid search

```
python -m src.eval.grid --ks 5 6 --ngram-ranges 1,1 4,4 --n-splits 3
```

✓ **Expected:** runs 4 grid cells, prints sorted results, saves `grid_search_report.json`.

Test 3.8 — DNABERT-2 frozen embeddings

```
python -m src.ml.dnabert.embed --source raw
```

✓ **Expected:** encodes ~766 sequences in ~1 min on GPU, saves `models/dnabert2_embeddings.npz`.

Test 3.9 — Train DNABERT-2 head

```
python -m src.ml.dnabert.train
```

✓ **Expected:** trains LR head, saves `models/dnabert2_head.pkl`, prints metrics.

Test 3.10 — DNABERT-2 single prediction

```
python -m src.ml.dnabert.predict  
"ATGGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAAATGTCATTAATGCTATGCAGAAAATCTTAGAGT"
```

✓ **Expected:** JSON with `predicted_class` + `probabilities` + `embedding_norm`.

Test 3.11 — DNABERT-2 fine-tune

```
python -m src.ml.dnabert_finetune --epochs 1 --batch-size 2 --grad-accum 8 --  
samples-per-class 200
```

✓ **Expected:** completes in ~2 min on RTX 3050, saves `models/dnabert2_finetuned/`. Use longer epochs/samples for real training.



Part 4 — Edge Case Tests

Test 4.1 — Empty sequence

```
$body = @{ sequence = "" } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/predict -ContentType
"application/json" -Body $body
```

✓ **Expected:** HTTP 400 with detail "No valid DNA bases found." or similar.

Test 4.2 — Very long sequence (10k bases)

📖 Generate a 10,000-base random sequence and submit it via the UI or API.

✓ **Expected:** prediction succeeds (sliding-window inference for DNABERT-2). May take a few seconds.

Test 4.3 — Mixed-case input

```
$body = @{ sequence = "atgGATtttATC" } | ConvertTo-Json # too short, just to
check case handling
```

✓ **Expected:** preprocessor uppercases it; if length OK, predicts normally.

Test 4.4 — Sequence with N bases

```
$body = @{ sequence =
"ATGNNNGATTTATCTGCTCTTCGCGTTGAAGAAGTACAAATGTCATTAATGCTATGCAGAAAATCTTAGAGTGTCCCAT
CTGTCTG" } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/predict -ContentType
"application/json" -Body $body
```

✓ **Expected:** `n_content > 0` in stats, prediction still works.

Test 4.5 — Concurrent requests

📖 Open 3 browser tabs, click Analyze on all of them within a second.

✓ **Expected:** all 3 succeed (FastAPI handles concurrency); responses arrive within a few seconds each.

Test 4.6 — Server restart while UI open

📖 Stop the server, refresh the browser.

✓ **Expected:** sidebar shows red "API offline". Restart server → green "API online" within 1 second.



Part 5 — Environment & Setup Tests

Test 5.1 — Verify Python interpreter

```
venv\Scripts\python.exe --version
```

✓ **Expected:** Python 3.10.x

Test 5.2 — Verify CUDA + PyTorch

```
venv\Scripts\python.exe -c "import torch; print('cuda:',  
torch.cuda.is_available(), 'device:', torch.cuda.get_device_name(0) if  
torch.cuda.is_available() else 'cpu')"
```

✓ **Expected:** cuda: True device: NVIDIA GeForce RTX 3050 ... on a GPU machine.

Test 5.3 — Verify all dependencies

```
venv\Scripts\python.exe -c "import fastapi, sklearn, transformers,  
sentence_transformers, faiss, reportlab, biopython, google.genai; print('all  
good')"
```

✓ **Expected:** prints all good.

Test 5.4 — Verify .env loads

```
venv\Scripts\python.exe -c "from dotenv import load_dotenv; load_dotenv(); import  
os; print('GEMINI:', 'set' if os.getenv('GEMINI_API_KEY') else 'missing')"
```

✓ **Expected:** GEMINI: set if you've configured the key.



Summary Checklist

After running through this guide, you should have ticked off:

- ☐ All 8 UI pages working

- ☐ All 4 disease classes predicting correctly
- ☐ All 3 model backbones return results
- ☐ OOD / similarity / saliency / ORF / circular plot all render
- ☐ PDF download works
- ☐ Mutation Lab compares correctly with effect classification
- ☐ Batch FASTA upload processes multiple records
- ☐ Copilot returns answers in DEMO mode
- ☐ Copilot returns answers in GEMINI mode (if key set)
- ☐ Source chips visible in chat
- ☐ Dark mode toggles across all pages
- ☐ Presentation slides navigate with keyboard
- ☐ All 12 JSON API endpoints respond correctly
- ☐ CLI commands for predict / audit / build / train all work
- ☐ Edge cases (invalid, short, long, N-bases) handled gracefully

If everything ticks ☒, you're production-ready.

Last updated alongside the GenomeIQ Step 7 documentation pass.